

Universidad Técnica de Machala

Documentación del Proyecto

Dashboard de Seguridad en Casa

Sistema de alarma, sensores PIR, control de accesos y ventanas con visualización digital twin
3D y notificaciones por Telegram

Autoras:

Debbie Milena Aguilar Lima

Melissa Yamileth Garrido López

Materia: Internet de las Cosas

Índice

1. INTRODUCCIÓN	3
2. OBJETIVO	3
3. REQUERIMIENTOS TÉCNICOS	3
4. METODOLOGÍA EMPLEADA	4
5. DESCRIPCIÓN DETALLADA DEL FLUJO	5
6. IMÁGENES DEL DASHBOARD	10
7. CONCLUSIONES	12
8. BIBLIOGRAFÍA	13

1. INTRODUCCIÓN

El presente proyecto tiene como propósito desarrollar un dashboard interactivo de seguridad residencial utilizando Node-RED como plataforma principal. En el contexto actual, la domótica y los sistemas de vigilancia inteligente han cobrado gran relevancia, ya que permiten a los usuarios monitorear y controlar el estado de su hogar de forma centralizada y en tiempo real.

Este sistema integra múltiples subsistemas: un panel de alarma con cuatro modos de operación, sensores PIR para detección de movimiento en dormitorios, cocina y sala, control de accesos para la puerta principal y ventanas, un visualizador de riesgo dinámico, un gemelo digital 2D superpuesto sobre una imagen isométrica de la casa, un visor 3D interactivo del modelo de la vivienda, un historial de eventos en tiempo real, y notificaciones automáticas por Telegram. Toda la lógica se implementó mediante nodos change, function y delay dentro de Node-RED, sin depender de bases de datos externas, usando el contexto global como almacén de estado.

2. OBJETIVO

El objetivo principal del proyecto es diseñar y desarrollar un dashboard de seguridad residencial funcional, atractivo y centrado en el usuario, que permita:

- Monitorear en tiempo real el estado de la puerta principal y de las ventanas de sala y cocina.
- Detectar movimiento en las zonas de dormitorios, cocina y sala mediante sensores PIR simulados.
- Controlar el sistema de alarma en cuatro modos: Activada, Desactivada, Modo Casa y Emergencia.
- Calcular y visualizar un índice de riesgo dinámico (0–100 %) según el estado de todos los sensores y accesos.
- Registrar un historial cronológico de los últimos 30 eventos del sistema.
- Visualizar el estado de la vivienda mediante un gemelo digital 2D y un modelo 3D interactivo.
- Enviar notificaciones automáticas al bot de Telegram 'SeguridadCasaBot' cuando cambia el estado del sistema.

3. REQUERIMIENTOS TÉCNICOS

Hardware

- Computador personal con capacidad para ejecutar Node.js y Node-RED de forma local.
- Conexión a Internet para acceder al dashboard desde el navegador y para el envío de mensajes a Telegram.

Software

- Node.js: entorno de ejecución JavaScript base para Node-RED.
- Node-RED: plataforma de flujo visual para IoT e integración de sistemas.

Paletas (nodos) instaladas en Node-RED

- @flowfuse/node-red-dashboard v1.30.2 — provee los nodos ui-button, ui-switch, ui-template y ui-group usados en la interfaz.
- node-red-dashboard v3.6.6 — compatibilidad con nodos ui_tab y ui_base legacy.
- node-red-contrib-telegrambot v17.4.12 — permite conectar Node-RED con un bot de Telegram para el envío de alertas.

Requisitos adicionales

- Token del bot de Telegram ('SeguridadCasaBot') configurado en el nodo 'telegram bot'.
- Modelo 3D de la casa en formato. glb alojado en GitHub (repositorio xpolaca/Casa3D).
- Imagen isométrica de la vivienda alojada externamente (image2url.com) para el gemelo visual 2D.
- Navegador moderno con soporte para Vue.js (integrado en FlowFuse Dashboard) y WebGL para el visor 3D.

4. METODOLOGÍA EMPLEADA

El desarrollo del proyecto siguió una metodología iterativa por módulos, definiendo primero la arquitectura del estado global y construyendo luego cada subsistema de forma incremental.

1. Definición de la arquitectura de estado

Se diseñó un objeto homeState almacenado en el contexto global de Node-RED, que contiene todas las variables del sistema: doorMain, doorBack, winSala, winCocina, motionSala, motionCocina, motionDormitorio, alarm, history y risk. Este objeto actúa como fuente única de verdad para todos los nodos.

2. Instalación y configuración del entorno

Se instalaron las paletas requeridas: @flowfuse/node-red-dashboard para la UI moderna, node-red-dashboard para compatibilidad y node-red-contrib-telegrambot para las alertas. Se configuró el bot 'SeguridadCasaBot' y se vinculó al chat ID del receptor.

3. Diseño del dashboard

Se definió una página Dashboard Seguridad en Casa con tema oscuro (Neon Dashboard Theme) y seis grupos: Estado General (Riesgo), Visualización Gemelo Digital, Sistema de Alarma, Sensores PIR, Control Accesos y Ventanas, y Modelo 3D Interactivo.

4. Implementación de los flujos

Se crearon grupos lógicos de nodos change encadenados: cada acción del usuario (botón o switch) dispara un nodo change que actualiza el campo correspondiente en homeState, luego un nodo

change central genera el historial, y finalmente otro nodo calcula el riesgo y propaga el payload a todos los widgets del dashboard.

5. Pruebas y validación

Se probó cada subsistema individualmente (alarma, PIR, puertas, ventanas) y luego en conjunto, verificando que el índice de riesgo, el historial, el gemelo digital y las notificaciones Telegram respondieran correctamente a cada evento.

6. Optimización

Se añadió un nodo delay 'Throttle 10s' para limitar los mensajes de Telegram a un máximo de uno cada 10 segundos, evitando spam al bot ante cambios de estado rápidos.

7. Documentación y entrega

Se documentaron los flujos, se exportó el archivo flows.json y se preparó esta documentación técnica.

5. DESCRIPCIÓN DETALLADA DEL FLUJO

El flujo completo se organiza en una sola pestaña llamada 'Seguridad en Casa', con tres grupos lógicos en el editor y seis grupos de UI en el dashboard.

5.1.Vista general del flujo en Node-RED

En la Figura 1 se observa la disposición completa del flujo. Todos los nodos de entrada (botones y switches) convergen hacia nodos change que actualizan el contexto global, luego pasan por la lógica de historial y cálculo de riesgo, y finalmente distribuyen el payload actualizado a los cuatro componentes de visualización del dashboard y al bot de Telegram.

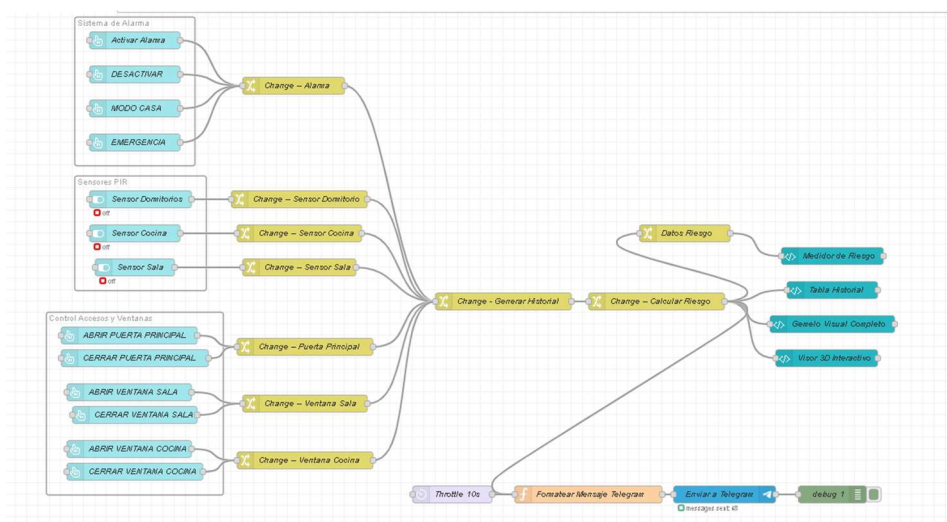


Figura 1: Captura completa del editor Node-RED mostrando todos los nodos del flujo.

5.2. Sistema de Alarma

En la Figura 2 se muestra el grupo 'Sistema de Alarma'. Contiene cuatro botones ui-button que emiten distintos valores de string como payload con topic 'alarm':

- ACTIVAR ALARMA: payload: 'activada'
- DESACTIVAR: payload: 'desactivada'
- MODO CASA: payload: 'casa'
- EMERGENCIA: payload: 'emergencia'

Cada botón conecta al nodo change 'Change – Alarma', que primero inicializa homeState si no existe (con JSONata) y luego escribe el valor del payload en homeState.alarm del contexto global, pasando el flujo al nodo central de historial.

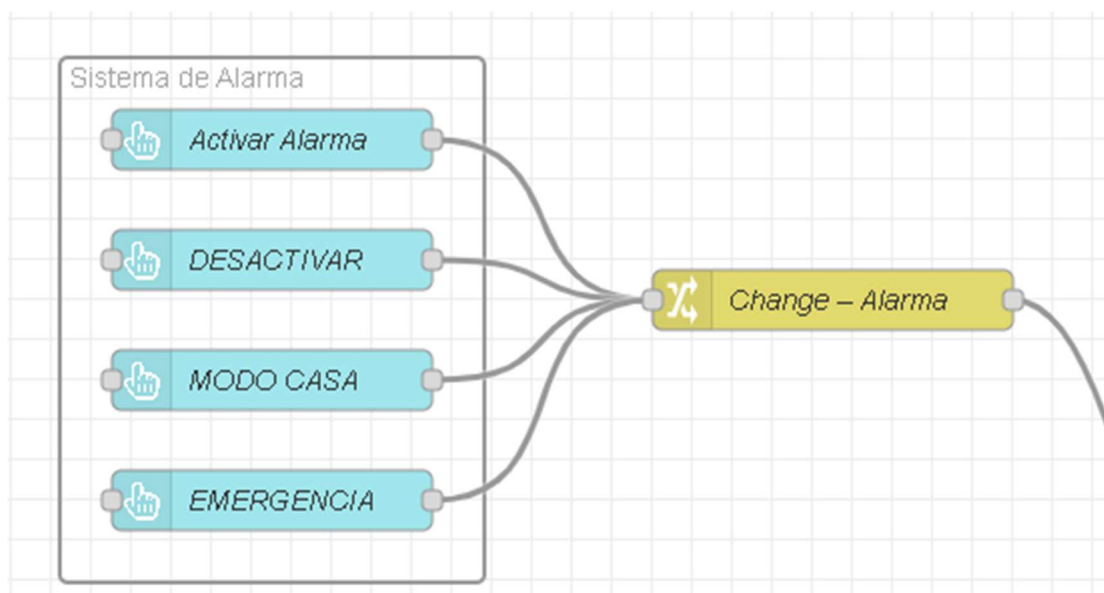


Figura 2: Grupo 'Sistema de Alarma' — cuatro botones conectados al nodo Change – Alarma.

5.3. Sensores PIR

En la Figura 3 se muestra el grupo 'Sensores PIR'. Contiene tres nodos ui-switch que simulan sensores de movimiento (PIR) en distintas zonas de la vivienda:

- Sensor Dormitorios: topic: 'motionDormitorio', valores: true/false
- Sensor Cocina: topic: 'motionCocina', valores: true/false
- Sensor Sala: topic: 'motionSala', valores: true/false

Cada switch se conecta a su propio nodo change (Change – Sensor Dormitorio, Change – Sensor Cocina, Change – Sensor Sala) que actualiza el campo correspondiente en homeState del contexto global.

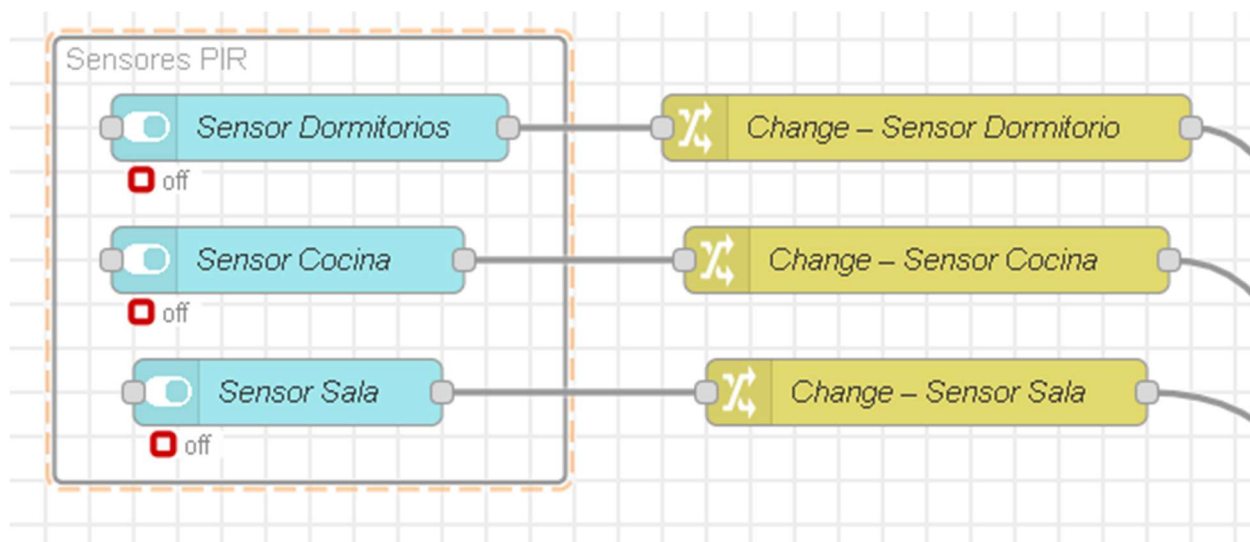


Figura 3: Grupo 'Sensores PIR' — tres switches con sus nodos change respectivos.

5.4. Control de Accesos y Ventanas

En la Figura 4 se muestra el grupo 'Control Accesos y Ventanas'. Incluye seis botones ui-button organizados en tres pares abrir/cerrar:

- ABRIR / CERRAR PUERTA PRINCIPAL □ topic: 'doorMain', payload: true/false
- ABRIR / CERRAR VENTANA SALA □ topic: 'winSala', payload: true/false
- ABRIR / CERRAR VENTANA COCINA □ topic: 'winCocina', payload: true/false

Cada par de botones conecta a un nodo change dedicado (Change – Puerta Principal, Change – Ventana Sala, Change – Ventana Cocina) que actualiza el campo boolean correspondiente en homeState.

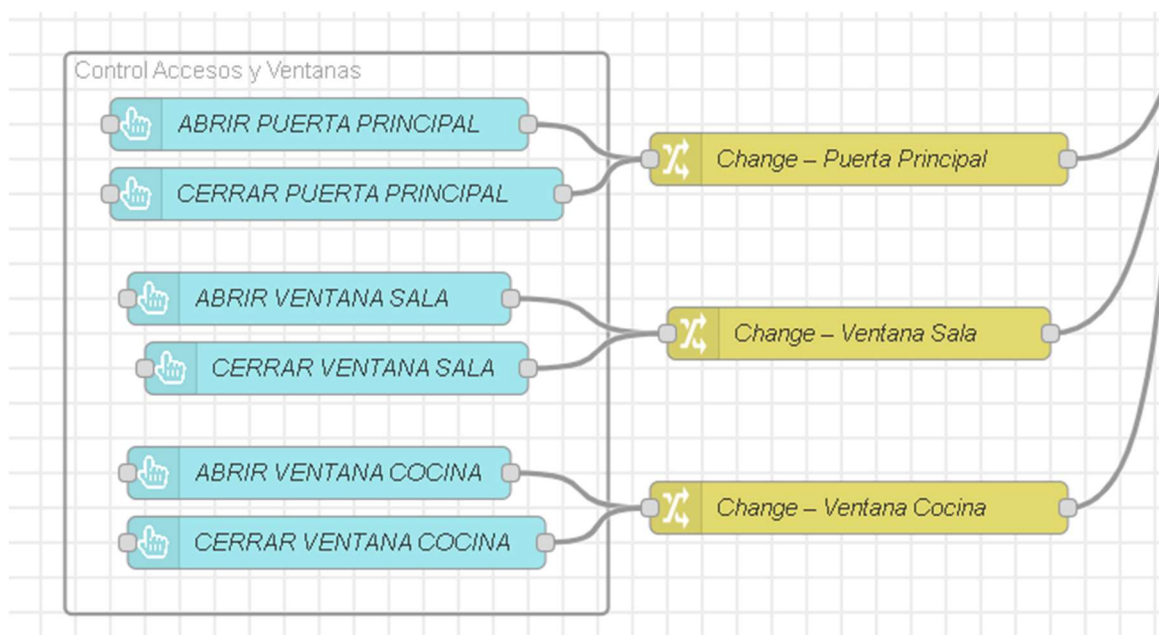


Figura 4: Grupo 'Control Accesos y Ventanas' — seis botones con nodos change.

5.5. Lógica Central: Historial y Cálculo de Riesgo

En la Figura 5 se muestra la columna central del flujo, donde convergen todos los nodos change de los subsistemas anteriores.

Change – Generar Historial: Este nodo recibe el mensaje de cualquier subsistema y, usando JSONata, construye un objeto de entrada al historial con tres campos: time (hora actual en formato HH:MM:SS ajustada a UTC-5), event (descripción legible del evento según el topic, por ejemplo 'Ventana Sala Abierta') y rawDevice (tipo de dispositivo: Cerradura, Magnético, Sensor PIR o Panel Central). El nuevo evento se antepone al array existente, manteniendo un máximo de 30 entradas.

Change – Calcular Riesgo: A continuación, este nodo lee el homeState completo del contexto global y calcula el índice de riesgo sumando puntos por cada condición activa:

Condición	Puntos
Puerta principal abierta (doorMain)	20
Puerta trasera abierta (doorBack)	20
Ventana sala abierta (winSala)	10
Ventana cocina abierta (winCocina)	10
Movimiento en sala (motionSala)	15
Movimiento en cocina (motionCocina)	15
Movimiento en dormitorio (motionDormitorio)	15
Movimiento en patio (motionPatio)	15
Alarma activada (alarm = 'activada')	10
Emergencia (alarm = 'emergencia')	100 (máx)

El resultado se limita a 100 mediante la función $\$min([suma, 100])$ de JSONata y se guarda en homeState.risk. Finalmente el payload completo (todo homeState con risk actualizado) se envía a los cuatro componentes de visualización y al nodo de Telegram.

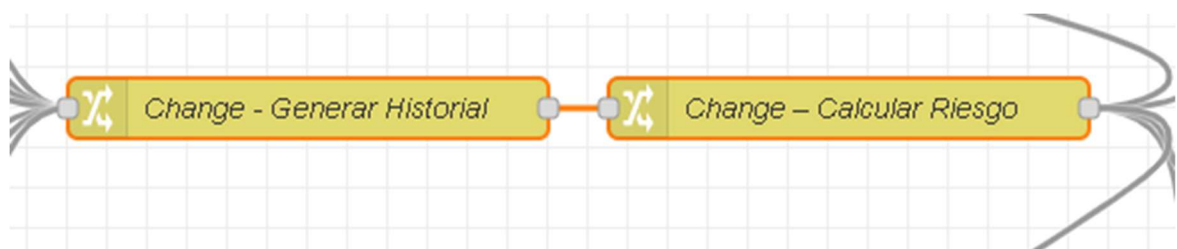


Figura 5: Nodos centrales Change – Generar Historial y Change – Calcular Riesgo.

5.6. Componentes de Visualización del Dashboard

El nodo 'Change – Calcular Riesgo' conecta en paralelo a cuatro destinos:

a) Medidor de Riesgo (ui-template)

Template Vue.js que muestra un círculo dinámico con el porcentaje de riesgo. El color del borde y del número cambia automáticamente: verde (#22c55e) para riesgo menor al 30 %, amarillo (#f59e0b) entre 30 % y 70 %, y rojo crítico (#a32126) por encima del 70 %. Debajo del círculo aparece el texto RIESGO BAJO, RIESGO MEDIO o RIESGO CRÍTICO según corresponda.

b) Tabla Historial (ui-template)

Template Vue.js que renderiza los últimos 30 eventos del sistema en una tabla con columnas HORA, EVENTO y DISPOSITIVO. Los eventos de tipo alerta (puertas o ventanas abiertas, movimiento detectado, emergencia) se resaltan en rojo para facilitar la identificación visual. Si no hay eventos, muestra 'Esperando eventos...!.

c) Gemelo Visual Completo (ui-template)

Template Vue.js que superpone elementos visuales sobre una imagen isométrica de la casa. Muestra etiquetas tipo 'glassmorphism' en el centro del dashboard cuando la alarma está en modo emergencia, modo casa o activada. Además, dibuja efectos de resplandor pulsante sobre las habitaciones con movimiento detectado (cocina, sala, dormitorios) y neon-doors para las puertas (verde = cerrada, rojo = abierta con animación 3D CSS de rotación).

d) Visor 3D Interactivo (ui-template)

Template Vue.js que carga el componente estándar de Google (v4.0.0) para renderizar el modelo.glb de la casa con rotación automática, controles de cámara y sombras. Cuando un sensor o acceso se activa, la cámara hace zoom automático hacia la zona correspondiente (sala, cocina, dormitorio, puerta principal, ventana sala o ventana cocina) mediante animación suave. También muestra hotspots con anotaciones de color rojo sobre las zonas activas. En modo emergencia, la cámara se aleja para mostrar la casa completa con un overlay rojo pulsante.

5.7. Notificaciones por Telegram

En la Figura 6 se muestra el flujo de notificaciones. El nodo 'Change – Calcular Riesgo' envía el payload también a un nodo delay 'Throttle 10s' (1 mensaje máximo cada 10 segundos, descartando excedentes). El flujo continúa al nodo function 'Formatear Mensaje Telegram', que construye un mensaje en formato Markdown con emojis, una barra de progreso de texto del riesgo, el estado de la alarma, el estado de

todos los accesos (puerta principal, trasera, ventanas) y el estado de los sensores PIR. El mensaje se envía al bot 'SeguridadCasaBot' (chat ID: 5209094665) mediante el nodo 'telegram sender'.

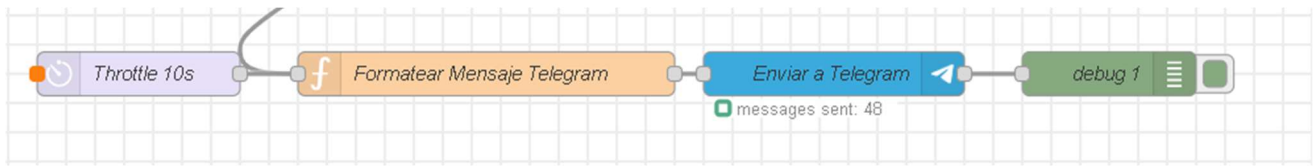


Figura 6: Flujo de Telegram — Throttle 10s → Formatear Mensaje → telegram sender.

6. IMÁGENES DEL DASHBOARD

Dashboard en funcionamiento — vista superior:

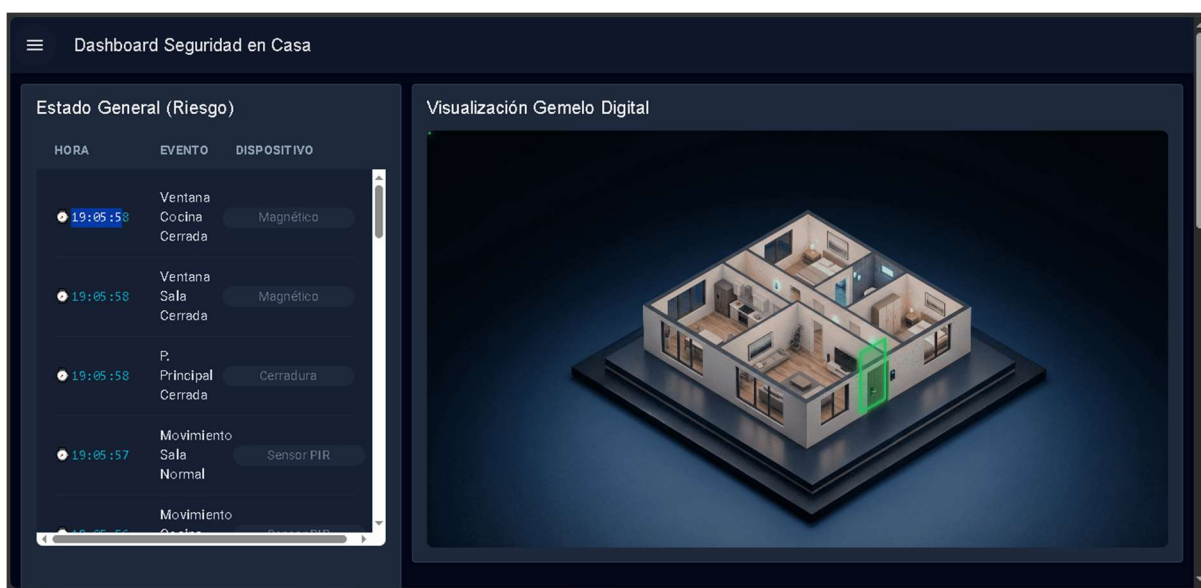


Figura 7: Vista superior del dashboard mostrando el historial de eventos y el gemelo digital 2D.

Dashboard en funcionamiento — vista central (medidor y controles):

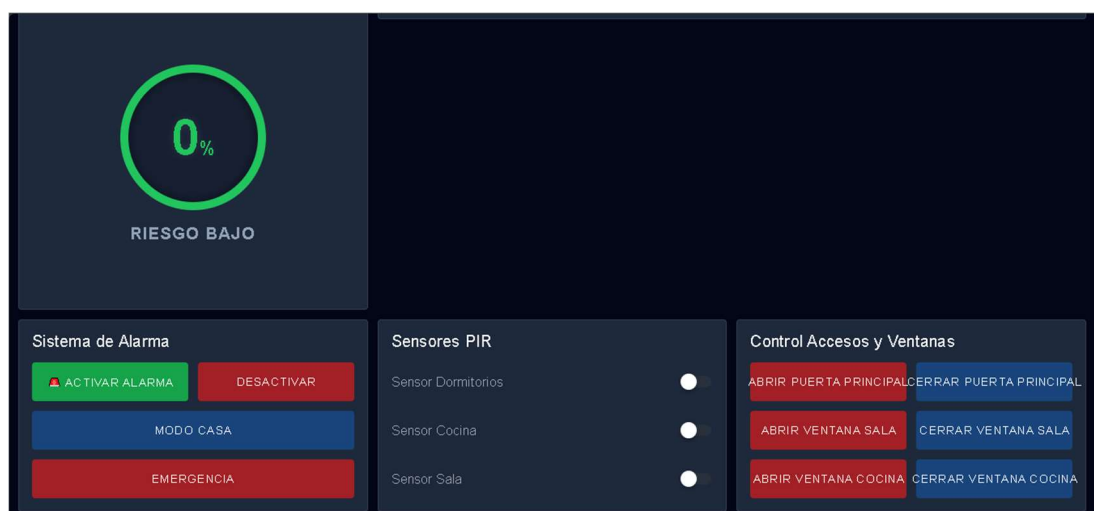


Figura 8: Vista central con el medidor de riesgo (0% RIESGO BAJO), sistema de alarma, sensores PIR y control de accesos.

Dashboard en funcionamiento — vista del modelo 3D interactivo:



Figura 9: Vista inferior con el modelo 3D interactivo de la vivienda cargado en model-viewer.

Notificaciones en Telegram — bot SeguridadCasaBot:

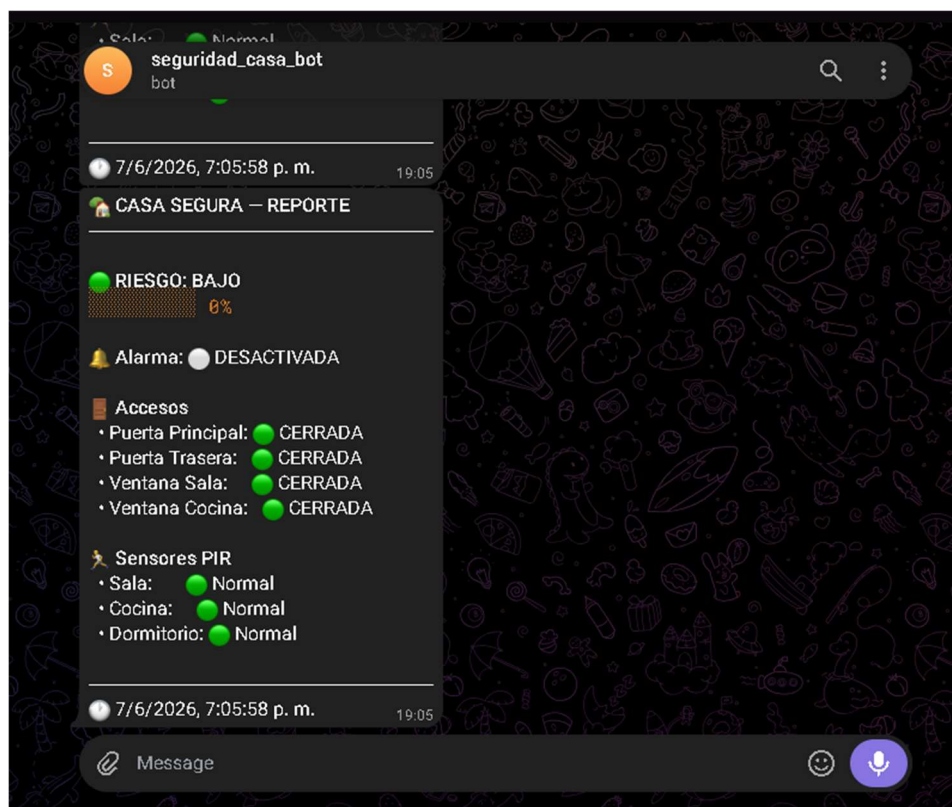


Figura 10: Mensaje enviado por el bot al chat de Telegram mostrando riesgo, alarma, accesos y sensores.

7. CONCLUSIONES

Funcionalidad alcanzada: El dashboard desarrollado en Node-RED demostró ser una herramienta eficiente para centralizar el monitoreo y control de un sistema de seguridad residencial, integrando sensores PIR, control de accesos, sistema de alarma con cuatro modos y notificaciones por Telegram de manera coherente y en tiempo real.

Arquitectura de estado centralizada: El uso del contexto global de Node-RED como almacén único de estado (`homeState`) simplificó significativamente la sincronización entre los distintos subsistemas, evitando inconsistencias y facilitando la trazabilidad de los cambios a través del historial de eventos.

Visualización avanzada: La implementación del gemelo digital en dos capas —un visor 2D superpuesto sobre imagen isométrica y un modelo 3D interactivo con `model-viewer`— permitió ofrecer una representación visual intuitiva del estado de la vivienda, diferenciando claramente las zonas en alerta mediante animaciones CSS y efectos de color.

Cálculo de riesgo dinámico: El índice de riesgo calculado mediante JSONata proporciona una métrica cuantitativa inmediata del nivel de seguridad de la vivienda, respondiendo automáticamente a cualquier cambio de estado sin necesidad de lógica adicional en el servidor.

Integración con Telegram: El bot '`SeguridadCasaBot`' con throttling de 10 segundos garantiza que el usuario reciba alertas oportunas sin saturar el canal de mensajes, demostrando que Node-RED puede integrarse eficientemente con servicios de mensajería externos para notificaciones IoT.

Potencial de expansión: El sistema está diseñado de forma modular, lo que permite agregar fácilmente nuevos sensores, cámaras, actuadores físicos (cerraduras MQTT) o integrar una base de datos para persistencia histórica, sin necesidad de rediseñar la arquitectura existente.

8. BIBLIOGRAFÍA

- [1] Node-RED Project, "Node-RED Documentation," *nodered.org*, 2026. [En línea]. Disponible: <https://nodered.org/docs/>
- [2] FlowFuse Inc., "Node-RED Dashboard 2.0 Documentation," *dashboard.flowfuse.com*, 2026. [En línea]. Disponible: <https://dashboard.flowfuse.com/>
- [3] K. Kohlhaas, "node-red-contrib-telegrambot," *npmjs.com*, 2026. [En línea]. Disponible: <https://www.npmjs.com/package/node-red-contrib-telegrambot>
- [4] Google LLC, "model-viewer: 3D Model Viewer for the Web," *modelviewer.dev*, 2026. [En línea]. Disponible: <https://modelviewer.dev/>
- [5] JSONata Project, "JSONata — JSON Query and Transformation Language," *jsonata.org*, 2026. [En línea]. Disponible: <https://jsonata.org/>
- [6] OpenJS Foundation, "Node.js v22 Documentation," *nodejs.org*, 2026. [En línea]. Disponible: <https://nodejs.org/en/docs/>
- [7] Telegram Messenger LLP, "Telegram Bot API," *core.telegram.org*, 2026. [En línea]. Disponible: <https://core.telegram.org/bots/api>
- [8] Vue.js Contributors, "Vue.js 3 Documentation," *vuejs.org*, 2026. [En línea]. Disponible: <https://vuejs.org/guide/introduction>